Le génie pour l'industrie
**Département de génie logiciel et des TI**

UQÀM | Département d'informatique

# La gestion plus moderne des projets logiciels – chapitre 3

Professeur Alain April, ing. PHD
(avec la contribution de Sylvie Trudel, PhD, UQAM
Mis à jour par Bruel Gérançon, PhD, UQAM



*Le contenu de ce document est protégé par le droit d'auteur. Ce document peut être utilisé à des fins d'étude personnelle seulement. Aucune partie de cette œuvre ne peut être transmise ou reproduite d'une quelconque façon sans la permission du détenteur du droit d'auteur.*



# Chapitre 3: La gestion de projet classique d'un projet logiciel

**Gestion de projet plus moderne d'un projet logiciel**

- Les approches Agiles
- Le modèle de cycle de vie SCRUM
- Les trois rôles en SCRUM
- La planification et l'estimation de l'itération
- Le démarrage (l'itération 0)
- La réunion de planification d'itération
- La programmation en binôme
- Les mêlées quotidiennes
- La rencontre de rétrospective
- **Coordonner plusieurs équipes agiles d'un même projet**

# Introduction

- Imaginez que vous êtes gestionnaire d'un projet logiciel. Un client vous demande de développer une application Web de formation en ligne.

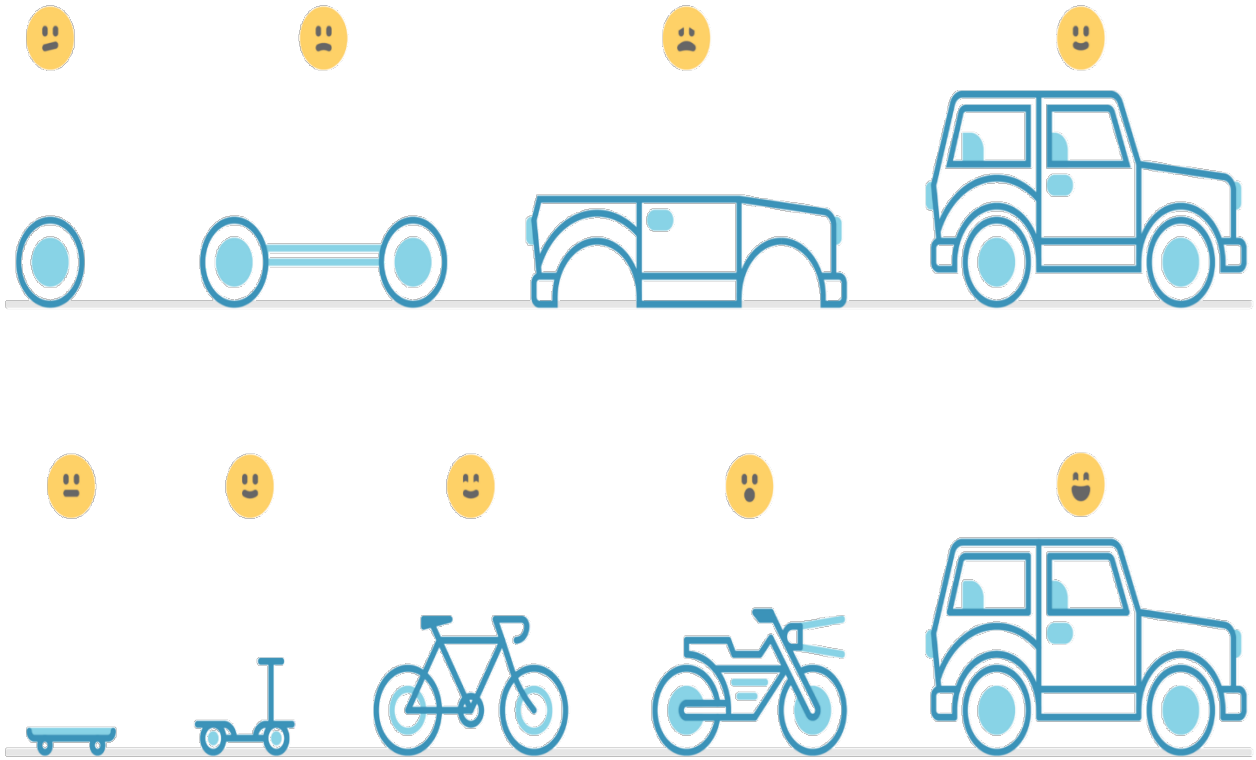
Vous lui dites :

- « Le projet prendra huit mois. Nous allons d'abord analyser tous vos besoins, ensuite concevoir le système, puis programmer l'ensemble de l'application et, à la fin, effectuer les tests. Vous verrez le produit complet dans huit mois. »
- Après six mois de travail, le client vous annonce :
  - « Mes besoins ont changé. Je veux maintenant une application accessible sur mobile, un chatbot pour guider les apprenants et de nouvelles fonctionnalités auxquelles je n'avais pas pensé au début. »

# Introduction

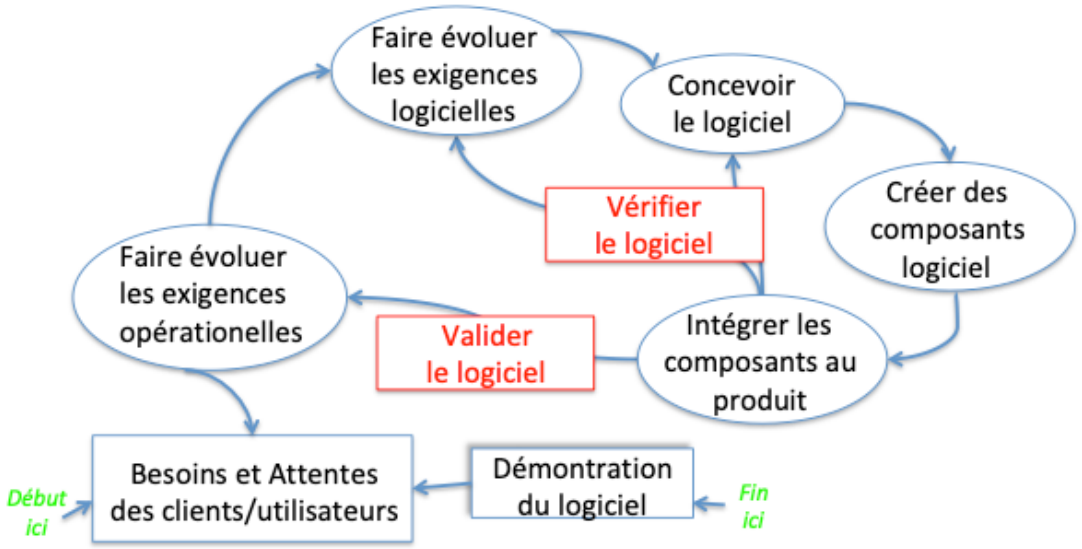
« Et si, au lieu d'attendre huit mois pour montrer le produit au client, nous lui avions livré une première version fonctionnelle après quelques semaines? »

# L’agilité en image



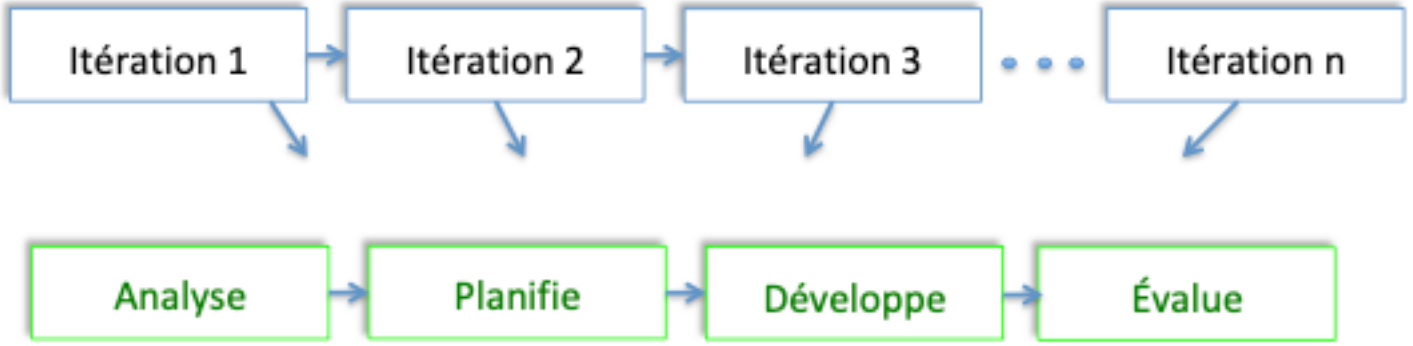
# La gestion plus moderne des projets

- Diagramme de vue d'ensemble d'un cycle de développement itératif typique:



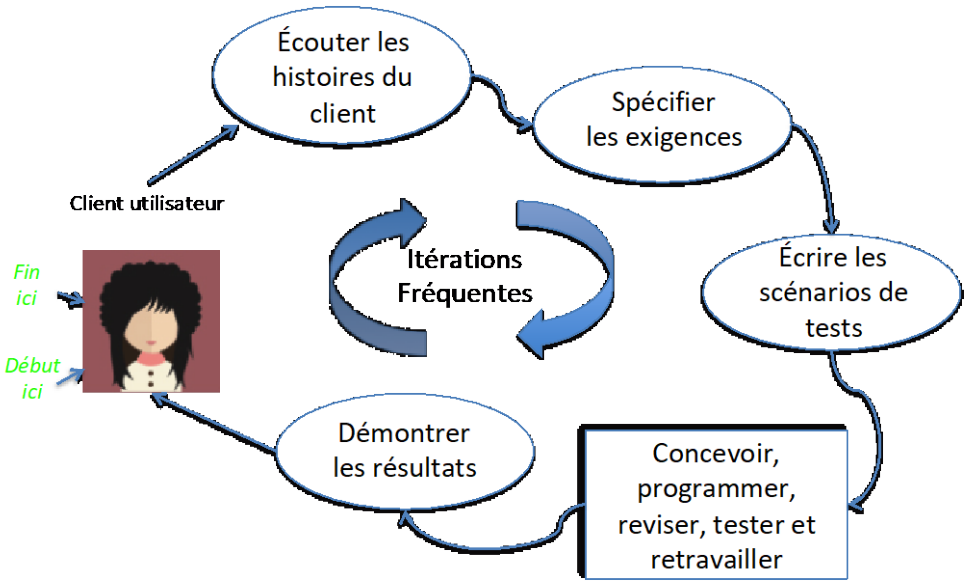
# La gestion plus moderne des projets

- Enjeux du chef de projet dans l'itératif:
  - Avoir un client impliqué pour pas travailler pour rien;
  - Faire des démos fréquemment
  - Prévenir rapidement quand il y a des problèmes
  - Savoir quand c'est d'assez bonne qualité pour être mis en production
- Perception du gestionnaire de projet:



# 3.2 - Introduction

- Agile:
  - Avoir un client impliqué;
  - Développer des cas de tests avant ou en même temps
  - Démos au lient
  - Livraison périodique en production
- Agile = Itératif:



# 3.2 – Planification Agile

- Quoi mettre dans votre plan de projet lors de la planification d'un projet Agile:
  - Le plan de projet initial doit spécifier le type de modèle itératif à utiliser;
  - il doit être adapté aux besoins du projet;
  - La durée de chaque itération doit être spécifiée dans le plan de projet initial;
  - Les principales activités et les principales étapes du projet doivent être identifiées et incluses dans le plan de projet initial;
  - À mesure que le produit évolue, les plans sont révisés et élaborés dans le respect des contraintes générales du projet;
  - Plusieurs activités de travail et plusieurs types d'activités de travail sont effectués simultanément;
  - Le contrôle de version automatisé est essentiel pour établir et gérer les lignes de base de divers produits de travail à différentes étapes de développement;
  - Une vérification et une validation indépendantes et itératives sont nécessaires;
  - L'alerte précoce des problèmes doit être traitée dès que détectée;
  - Les raisons de la reprise excessive du produit de base en croissance doivent être identifiées et des corrections apportées dès que possible;
  - Les développeurs, les utilisateurs, les clients, l'acquéreur et les autres parties prenantes concernées doivent faire l'objet de démonstrations fréquentes des progrès réalisés.

# Mise en contexte: les acteurs d'un projet logiciel



# Qu'est-ce que l'Agilité?

- c'est une approche de développement de logiciels
- Définie par le Manifeste Agile (2001)
  - 4 valeurs
  - 12 principes
- Mise en œuvre par un ensemble de méthodes, dont:
  - SCRUM
  - TDD et ATDD
  - eXtreme Programming (XP)
  - Etc.

# Les 4 valeurs de l'Agilité

- Les individus et leurs interactions plus que les processus et les outils
- Des logiciels opérationnels plus qu'une documentation exhaustive
- La collaboration avec les clients plus que la négociation contractuelle
- L'adaptation au changement plus que le suivi d'un plan

# Les 12 principes

**Valeur** d'affaires

**Accueillir** les **changements**

Cycles courts

Utilisateurs et développeurs **collaborent quotidiennement**

**Personnes motivées**

**Logiciel opérationnel** = LA mesure d'avancement

Communication **face-à-face**

**Rythme soutenable**

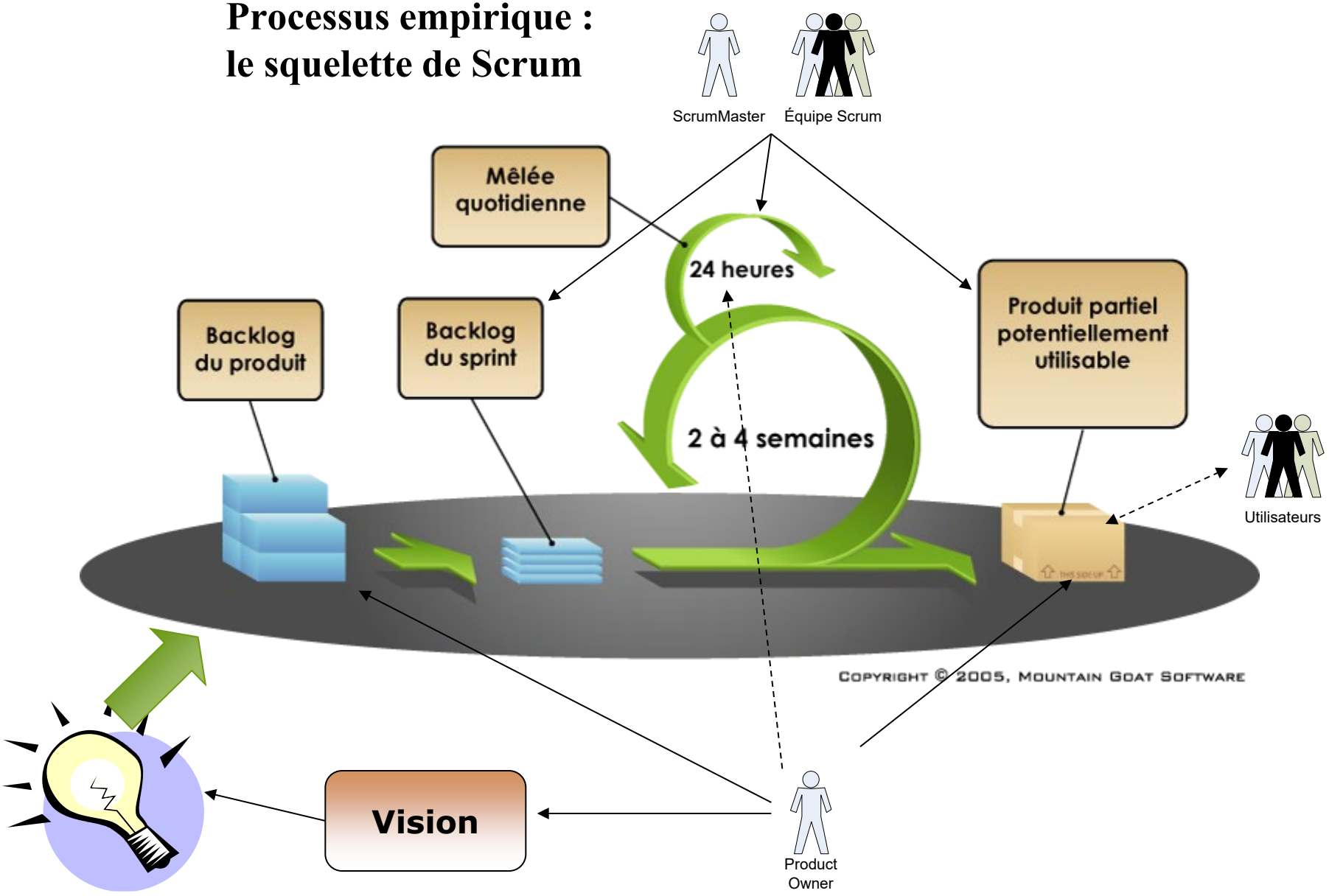
**EXCELLENCE TECHNIQUE**

Minimiser le **travail inutile**

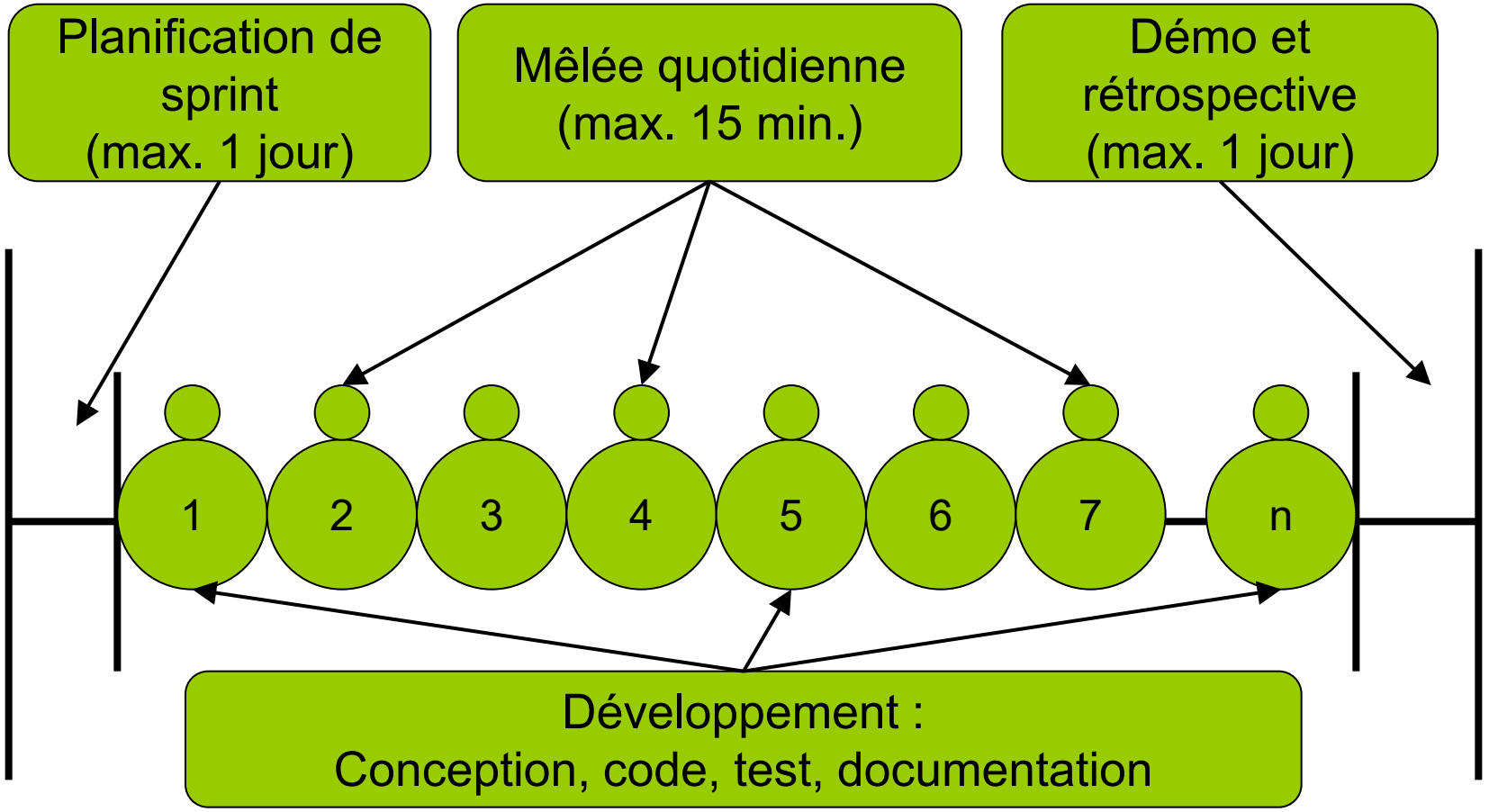
**Amélioration continue**

Architectures et conceptions émergent de l'équipe

# Processus empirique : le squelette de Scrum

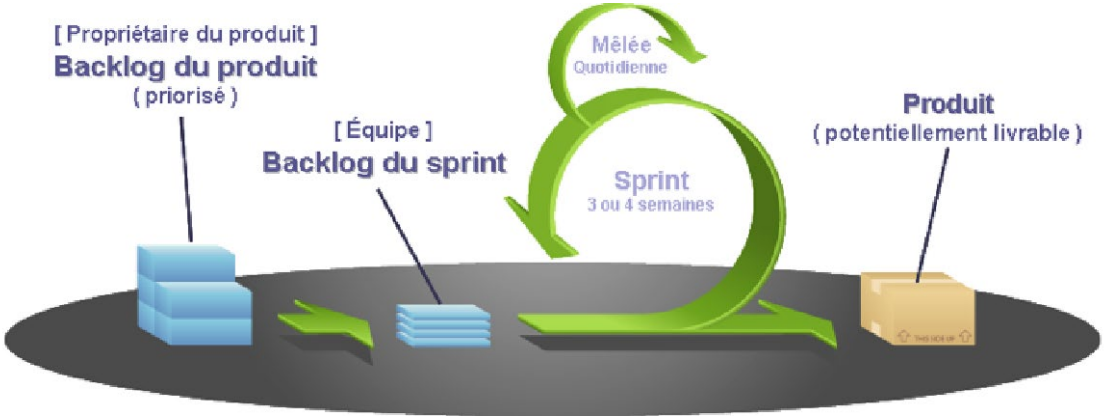


# Processus organisé en SPRINT




Sprint de 30 jours d'effort soutenu et ciblé (*focused*)

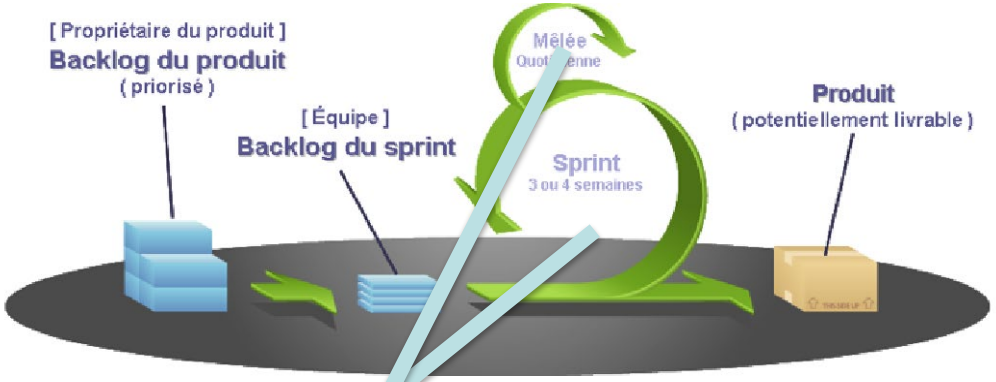
# 3.3 – SCRUM




Section 3.3.1, il y a 3 rôles dans cette méthodologie:

- **Le SCRUM master**: style de management participatif, coach, s'assure de la productivité (protège l'équipe des interférences externes) et du respect du processus, anime les rencontres;
- **Le/la propriétaire du produit**: porte la vision du produit, expert du domaine métier, rédige/explique les scénarios utilisateurs, émet ses priorités et les met dans le carnet de produit;
- **L'équipe de développement**: transforme les besoins en fonctionnalités utilisables, est pluridisciplinaire.

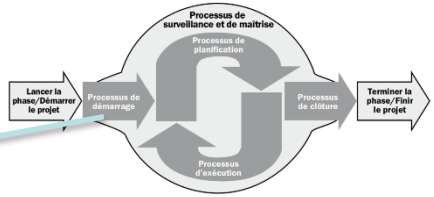
# 3.3 – SCRUM




## Section 3.3.1 : Mêlées et Itérations:

- **L'itération («Sprint»)**: d'une durée fixée par l'entreprise (varie de 7 jours à 30 jours calendrier) débute avec une rencontre de planification de l'itération, continue avec le travail de développement et la revue d'itération, et se termine par une réunion de rétroaction (appelée « rétrospective »);
- **La mêlée quotidienne**: une courte réunion de synchronisation des membres de l'équipe, debout d'un maximum de 15 minutes pour que chacun présente ce qu'il a réalisé hier (en lien avec le carnet de sprint), dire ce qu'il va faire aujourd'hui (toujours en lien avec le carnet de sprint), et exprimer s'il y a des obstacles qui ralentissent ou bloquent la progression.

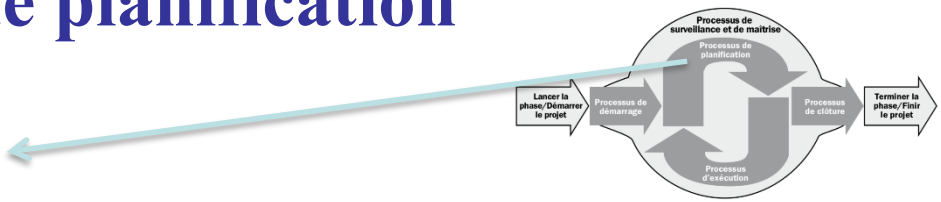
# 3.3.3 – SCRUM – Itération 0 (travaux préparatoires)




La planification des itérations (démarrage) – qui peut être plus long qu'une itération normale de développement:

- **Établir les priorités fonctionnelles**: Le/la propriétaire du produit va dicter les éléments du carnet de produit qui doivent être réalisés avant les autres;
- **Mettre en place/tester l'environnement de travail**: Le SCRUM board, la gestion de version sur GitLab (ou autre), les environnements (Dev, Test), les outils de tests,…;
- **Architecture logicielle et technologies**: l'équipe fait la conception de l'architecture haut niveau du «front-end et back-end», décide des technologies, cadriciels (*frameworks*), bibliothèques (*librairies*);
- **Concernant l'estimation**: les éléments du carnet de produit pour lesquels on a suffisamment d'informations (c-à-d. ceux qui étaient essentiels dans l'ordre de priorité) peuvent être estimés avec un assez grand niveau de confiance par l'équipe à ce stade-ci. Ensuite, on estime le reste des éléments du carnet par analogie.

# 3.3.4 – SCRUM – Première étape de chaque itération de développement – la réunion de planification




La réunion de planification:

- **Vérification que l'itération précédente est terminée**: Tous s'assurent que les travaux de l'itération précédente sont terminés;
- **Présentation du propriétaire de produit**: Fait une synthèse des objectifs, jalons et dates cibles et précise les tops « stories » du carnet de commandes (Quoi et Comment). Il précise quel devrait être le **but** de la prochaine itération;
- **L'équipe**: identifie les éléments du carnet de produit à inclure dans l'itération, confirme les estimations et le but du sprint (ce n'est pas un contrat ferme), chaque membre identifie ses responsabilités, découpe les « stories » en tâches (estimables en heures), met ses tâches dans le tableau SCRUM (doit tenir compte des disponibilités de l'équipe, soit de 70% à 80% de sa capacité pour se protéger des risques et obstacles, sinon le but du sprint ne tiendrait pas, ni le principe de « rythme soutenable »).

# Définition TERMINÉ

Quand une itération est à l'état « Terminé »

# Définir « terminé »

- La définition de « terminé »: la **capacité technique actuelle de l'équipe**
- La définition de « terminé » **devrait s'étendre** à toutes les activités nécessaires à la livraison en production
- Le travail qui n'est pas couvert pas la définition de « terminé » (travail « non terminé ») doit être identifié et porté au carnet de produit
- Tout élément qui ne respecte pas la définition de « terminé » **n'est pas présenté au directeur de produit** en fin de sprint

# Étendre la définition de « terminé »

- Conception révisée
- Code remanié
- Code révisé
- Tests unitaires
- Tests intégrés
- Tests de recette
- Tests de performance
- Tests d'ergonomie
- Documentation utilisateur
- Déploiement en pré-production
- Acceptation par les utilisateurs



# Définition terminé

**Définition de terminé** (Definition of Done – DoD) : c'est un résumé du plan d'assurance-qualité qui contient la liste des tâches qui doivent avoir été accomplies afin d'assurer la qualité d'une fonctionnalité développée.

Pour qu'une fonctionnalité soit considérée comme étant "Terminée", les éléments de processus suivants doivent avoir été appliqués adéquatement:

1. La fonctionnalité correspond en tous points aux maquettes approuvées;
2. L'adhésion aux normes de programmation a été vérifiée;
3. Les tests automatisés, qui couvrent au moins 75% du code, s'exécutent avec succès;
4. L'analyse statique du code avec SonarQube montre 0 mauvaises odeur (*code smells*), 0 bogues (*bugs*) et 0 vulnérabilités (*vulnerabilities*);
5. Le code a été soumis via une *Pull Request* (PR) qui a été fusionnée avec succès sur la branche *Master*;
6. Les tests d'acception (avec le PO) ont été effectués avec succès, à partir d'une version exécutable générée de la branche *Master*;
7. La documentation nécessaire a été faite dans les pages Wiki du projet, afin d'assurer la pérennité des informations pour les prochains développeurs;
8. La fonctionnalité répond correctement aux exigences demandées car elle est vérifiée/validée sur le serveur.

# Définition de prêt

- Définition de prêt VS Définition Terminé

# Définition de prêt

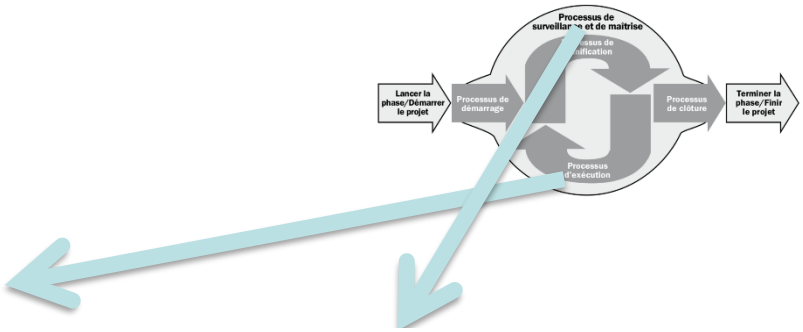
Pour qu'une fonctionnalité soit prête à être développée, nous devons atteindre une compréhension commune de ses exigences détaillées. Pour cela, les éléments suivants doivent avoir été accomplis:

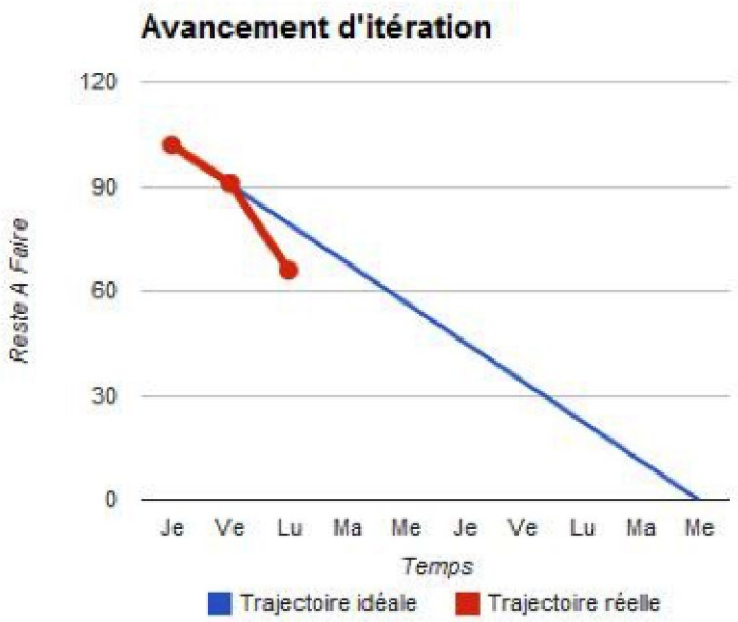
1. Les exigences détaillées ont été inscrites dans l'élément du carnet de produit:
    - Énoncé du récit utilisateur (*user story*) en 3 parties:
        - "En tant que <type d'utilisateur>"
        - "Je veux <action représentant la fonctionnalité>"
        - "Afin de <valeur d'affaires/bénéfice attendu pour cet utilisateur>"
    - Lien vers le prototype où des détails de fonctionnement sont définis ou insertion directe des images des prototypes;
    - Les critère d'acceptation (*Acceptance criteria*) sont définis, c'est-à-dire la liste des cas/scénarios de tests d'acception qui seront exécutés avec le PO lors de la démonstration de cette fonctionnalité;
    - Une priorité a été définie (1=Essentiel; 2=Important; 3=Souhaitable).
2. Le PO a révisé ces détails et a déplacé l'élément dans la colonne "Prêt à être développé".
3. Une conception détaillée initiale a été discutée entre 2 développeurs (ou plus) (classes, méthodes, API, structure de données, etc.);
4. Le développeur détermine la liste des tâches à faire dans la *User Story*, de sorte que l'élément puisse être "terminé" quand ces tâches auront été toutes accomplies (il sera possible d'en ajouter d'autres au besoin);
5. Le(s) développeur(s) ont estimé le niveau d'effort requis, en *User Story Points* (USP) ou en Points de Fonction Cosmic (PFC), en se basant sur les tâches à faire et en comparant avec les autres *User Stories*.

# 3.3.5 – SCRUM – Les activités d'itération

Pendant le sprint:

- **L'équipe**: s'assure d'effectuer ses tâches;
- **Le SCRUM Master**:
  - Fait le suivi de l'avancement avec l'équipe dans des mêlées quotidiennes: c.-à-d. graphe d'avancement d'itération;
  - Renonce à l'assignation des tâches: les développeurs choisissent leurs tâches – équipe autogérée (mais on la surveille, au cas où…)
  - S'assure que la somme des tâches qui complètent une « story » sont terminées (dev, tests, revues, prêtes pour la démonstration/déploiement) = éviter la surcharge de tests en fin d'itération;



# Équipe Scrum autogérée

- un responsable de produit
- un ScrumMaster
- des « équipiers »



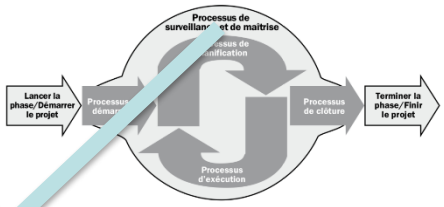
# Équipe Scrum autogérée

- S’auto-organise
- **Pluridisciplinaire** et ne comporte pas de rôles prédéterminés
- **Responsable de son engagement**
- Possède **l’autorité nécessaire** pour agir de manière à respecter ses engagements

# 3.3.6 La programmation en binôme

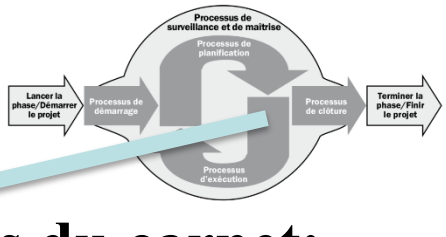
- Un binôme c'est: un pilote qui développe et un copilote qui fait les revues:
- **L'amélioration de la cohésion de l'équipe** : les développeurs ne travaillent plus seuls devant leur ordi, ils sont en contact permanent → pendant la durée du binôme de 1 à 4 heures par jour car c'est épuisant intellectuellement;
- **Le partage des connaissances** : il n'est pas rare d'avoir dans une équipe un développeur particulièrement doué. L'échange précédemment évoqué va tirer toute l'équipe vers le haut au travers du partage de connaissances;
- **Un meilleur code** : deux développeurs valent mieux qu'un. Le pilote peut se concentrer sur l'implémentation de l'algorithme, la gestion des contraintes et la validité du code
- Son "copilote" veille à ce que le projet aille dans la bonne direction et remplisse l'objectif premier.
- De plus, la revue du code en continu effectuée par le "copilote" va permettre d'obtenir un code optimal, souvent plus court et moins complexe;
- **Une forte capacité d'échange** : les deux développeurs peuvent échanger sur la meilleure voie d'implémentation et élucider la problématique pour mieux cerner les objectifs;
- **Une meilleure gestion du temps** : les points précédents devraient typiquement augmenter la qualité et permettre une implémentation de l'objectif plus rapidement.

# 3.3.7 Les mêlées quotidiennes




- **Stand-up de 15 minutes (lieu et heure fixe):**
  - Permet de comprendre  où l'on en est. Il faut que ce soit une activité de synchronisation du travail, pas seulement de «reporting»;
  - Chaque personne répond à 3 questions:
    - Qu'est-ce que j'ai fait hier dans l'objectif de finaliser une story?
    - Qu'est-ce que je vais faire aujourd'hui dans l'objectif de finaliser une story?
    - Quels obstacles me ralentissent ou me bloquent?
  - Le SCRUM master va:
    - Calculer la vélocité (c.-à-d. la productivité moyenne de l'équipe pour aider aux prochaines estimations)
    - Classer les obstacles, les éliminer au plus tôt

# 3.3.8 La revue de sprint




- **On vient confirmer la réalisation des éléments du carnet:**
  - L'équipe démontre au PO les éléments conformes à la définition de « terminé »
    - Si ce n'est pas « terminé » on ne le montre pas et on reporte sa complétion et sa démonstration au sprint suivant;
    - Typiquement, on exécute les tests d'acceptation de chaque élément terminé → ne devrait pas durer plus de 2 heures;
  - Le PO va formellement approuver ou pas chacun des éléments
    - Le PO peut aussi ajouter des éléments au carnet et les prioriser pour les prochains sprints, à la lumière de ce qu'il vient de constater;
  - Le SCRUM master va:
    - Noter les éléments approuvés ou pas de façon à confirmer la vélocité finale du sprint (soit en #USP/sprint, soit en #PFC/sprint)
    - S'assurer que les éléments non terminés seront pris en compte dans le prochain sprint planning.

# 3.3.9 La réunion de rétrospective

- **À la fin de l'itération:**




  - Examiner les résultats des améliorations précédente
  - Déterminer comment améliorer la situation ou le processus:
    - Relation clients, relation entre les développeurs
    - Interdépendances avec d'autres projets
    - Processus SCRUM
    - Problèmes technologiques (ceux qu'on n'a pas pu résoudre en cours de sprint)
  - Deux techniques populaires:
    - « Good »/ « Bad »/ « Ugly » → Nos bonnes pratiques (on en prend conscience), nos pratiques irritantes (elles sont faciles à régler), nos mauvaises pratiques (oh là là, celles-là doivent être réfléchies)
  - Le SCRUM master va:
    - Noter les améliorations à apporter;
    - Soutenir l'équipe à élaborer les solutions de ces améliorations afin qu'elles prennent effet tout de suite après.

# 3.3.10 Plus grands problèmes rapportés

- **Problèmes de SCRUM Masters:**
  - Les patrons pensent qu'un SCRUM master est un chef de projet, alors qu'il est au service de l'équipe;
  - La difficulté à établir des jalons de livrables (fixés dans le temps);
  - Gérer des équipes à distance → le face-à-face est dur à faire;
  - Les réunions s'éternisent → faut limiter les bloc de temps;
  - Gérer des demandes de changements urgentes qui ralentissent les développeurs → reflet d'un manque de vision ou mauvaise gestion des risques ou des priorités;
  - Manque d'autorité pour aider son équipe et éliminer les bloquants;
  - Abus de la méthodologie Agile → un prétexte pour aller plus vite sans vraiment suivre la philosophie de la méthode;
  - Aucune autorité de mettre à pied un client ou un développeur incompétent ou indiscipliné.

# Résumé des pratiques SCRUM

- 3 rôles:
  - *Product Owner* (PO)
  - Scrum Master
  - Membres d'équipe
- 4 pratiques (cérémonies)
  - Planification de sprint
  - Mêlée quotidienne
  - Revue de sprint (démo)
  - Rétrospective
- 4 livrables
  - Définition de terminé (sert de plan d'AQ)
  - Carnet de produit (backlog)
  - Carnet de sprint (liste d'activités)
  - Fonctionnalités

**Minimum!**

# Cas d’étude

**Gestion plus moderne des projets**

Cas #1- Vous développez votre projet en mode agile, et vous venez de commencer votre sprint. En cours de route, trois choses arrivent: i) pour implanter un cas d'utilisation/user story prévu pour le sprint actuel, vous vous rendez compte que vous devez en implanter un autre qui était prévu dans le plan de projet pour le sprint suivant, ii) une fonctionnalité implanté au sprint précédent s'avère erronée, nécessitant de l'effort de correction, et iii) votre client s'amène avec une nouvelle exigence prévue initialement pour le sprint suivant, parce qu'il doit faire une démonstration à un client potentiel à la fin du mois. Comment gérer tout cela?

# Étude de cas

Cas #2- Votre PO planifie une mise en production dès la fin de l'itération #1 dans le cadre de votre projet de développement. Quand vous l'annoncez à l'équipe, certains membres ont des doutes et expriment leur manque de confiance à l'application. Votre intuition vous indique alors que la date technique pourrait être plus importante que ce que l'équipe a laissé croire jusqu'à présent. Décrivez brièvement COMMENT vous allez procéder pour identifier et quantifier cette dette technique

# Étude des cas

Cas #3- Votre projet Agile, répartie en 3 itérations, est à sa 2e itération. L'estimation du "reste à faire" de l'équipe annonce un dépassement important du budget et de la date de livraison prévus. Dans ce contexte, discutez brièvement de trois (3) options possibles avec votre PO afin qu'il puisse prendre une décision éclairée.

# Étude des cas

Cas #4- Un gestionnaire de projet discute de l'importance de la qualité interne et de la qualité externe lors de la planification des sprints. Que devrait-on répondre à un PO qui essaie de faire réduire de moitié le nombre de points d'une User Story pendant la séance de la planification?

# Étude des cas

Cas #5- Vous êtes un VP d'une société, le CDP d'un projet difficile vient de livrer le 1er sprint de l'extrant. Or cette livraison est tardive et le PO menace d'évoquer pénalités pour retard lesquelles sont importantes. Le CDP explique que le retard est, selon lui, le temps excessif pris par le PO pour approuver le rapport le SPRINT. Comment présentez-vous ce sujet au comité de gestion de l'entreprise. Que ferez étant gestionnaire de projet afin d'éviter le retard dans les prochaines itérations.

# Étude des cas

Décrivez en vos mots l'importance d'une "définition de terminé" dans un projet de développement Agile, c'est-à-dire sa raison d'être. Et quelle serait une des conséquences de ne pas respecter cette définition de terminé.

# Kanban

- Méthode agile axée sur la visualisation et l'optimisation du flux de travail
- Méthode inspirée des méthodes manufacturières de compagnies tels que Toyota
- En livrant peu, mais fréquemment de manière à réduire l'inventaire du développement

# Agile à grande échelle

C’est quoi l’agile à grande échelle?

# Agile à grande échelle

- Appliquer des principes et pratiques Agile au sein d'organisations de grande taille, impliquant plusieurs équipes travaillant ensemble sur des projets complexes.
- Comment maintenir les avantages de l'Agilité  tout en gérant les défis liés à la coordination, la communication, et l'alignement entre plusieurs équipes?
    - Flexibilité, livraison rapide, amélioration continue

# 3.5 – Agile à grande échelle

- Il y a plusieurs méthodes pour coordonner le travail d'un ensemble d'équipes Agiles ensemble:
  - Les stand-up meetings diffèrent un peu:
    - Instaurer la livraison continue de toutes les équipes
    - Créer un Backlog commun à toutes les équipes
    - Favoriser une culture collaborative
    - Considérer l'utilisation d'une méthode à grande échelle:

# Agile à grande échelle

Le Scrum of Scrums

- Réunion de coordination utilisée dans le cadre de l'Agile à grande échelle pour synchroniser plusieurs équipes Scrum travaillant sur un projet commun.
- Représentants de chaque équipe se réunissent régulièrement (chaque jour ou chaque semaine)
  - Partage de  l'avancement des équipes.
  - Identification et résolution des interdépendances entre les équipes.
  - Traitement  des obstacles affectant les équipes.

# Cas pratique

- Comment utiliser l'agile à grande dans le cadre d'un projet de développement des Logiciels embarqués pour l'automobile où l'équipe de développement comporte 50 développeurs?

# Cas pratique (suite)

- Comment peut-on assurer l'intégration continue dans un tel contexte?
- Autrement dit comment tester et déployer rapidement les fonctionnalités?
  - En mettant en place des pipelines d'intégration et de déploiement continue